catalog · orders · notifications

CORSO · 24 ORE · 8 LEZIONI

Microservizi, Caching ed Event-Driven

Dal monolite a un mini e-commerce con gateway, cache ed eventi, costruito lezione dopo lezione.

Rocco Mazzeo
rocco.mazzeo@gmail.com

Fabio Rillo
fabiorillo.work@gmail.com

Cosa saprete fare a fine corso

01

Scomporre

Spezzare un monolite in servizi con responsabilità chiare e un database ciascuno.

02

Velocizzare

Usare Redis e la cache HTTP, sapendo quando invalidare e cosa può andare storto.

03

Disaccoppiare

Far comunicare i servizi con eventi su RabbitMQ, gestendo errori e duplicati.

E soprattutto: ragionare sui **trade-off**, cioè capire quando **non** usare tutto questo.

A chi è rivolto

Ragazzi post-diploma che hanno già scritto qualche programma e vogliono capire come si costruiscono i sistemi veri.

Non serve esperienza con Docker, cloud o database: si parte da zero su tutto questo.

PREREQUISITI

- Variabili, funzioni, cicli
- Un po' di HTTP e JSON
- Riga di comando di base

Chi parte da zero può seguire il modulo pre-corso opzionale di 2 ore.

La formula

24

ore in totale

8

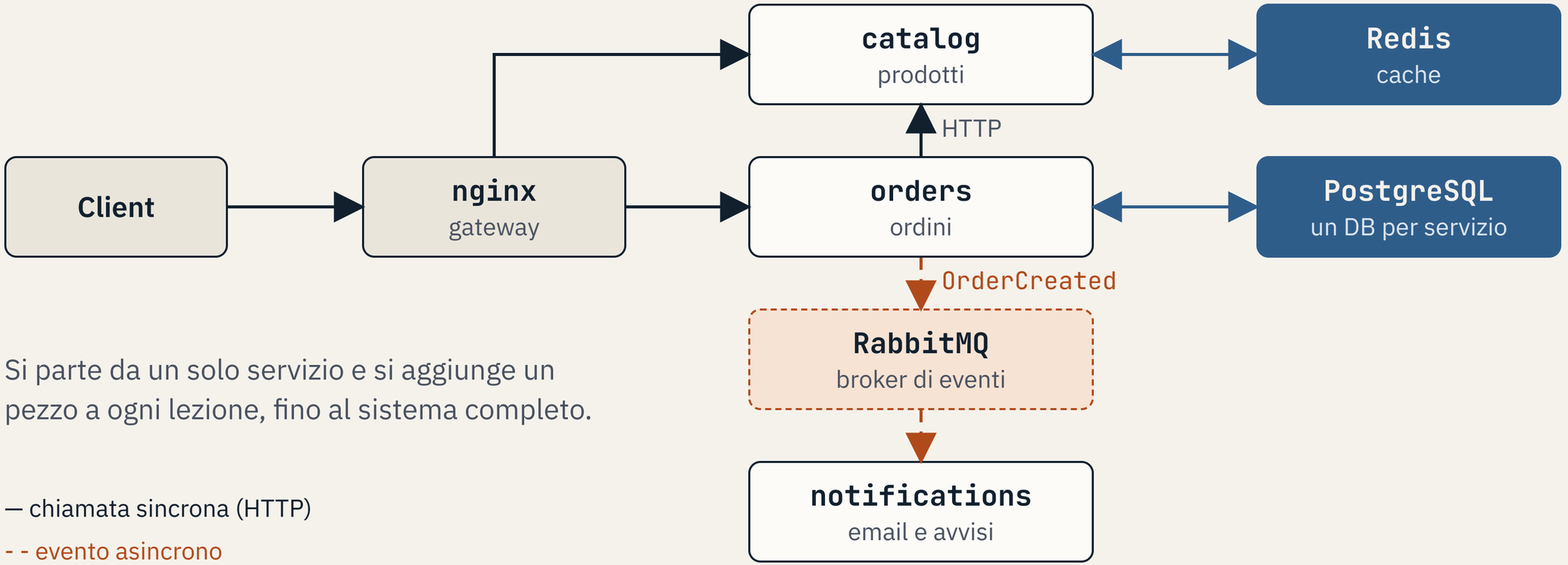
lezioni da 3 ore

40/60

teoria / laboratorio

Ogni lezione: teoria breve, laboratorio guidato sul progetto, e gli ultimi 10 minuti su una domanda: «cosa si rompe se...?»

Progetto fil rouge: mini e-commerce



Si parte da un solo servizio e si aggiunge un pezzo a ogni lezione, fino al sistema completo.

— chiamata sincrona (HTTP)

- - evento asincrono

Gli strumenti

Editor

VS Code con le estensioni Docker, REST Client e Dev Containers

Linguaggio

Node.js con TypeScript, framework Fastify o Express

Infrastruttura

Docker e Docker Compose, PostgreSQL, Redis, RabbitMQ, Nginx

Test e osservazione

curl e REST Client, k6 per i load test, RabbitMQ Management UI, redis-cli

Tutto gira in locale con docker compose up: nessun account cloud.

Il repository del corso

github.com/mg-i-roc-maz/corsi

Un solo repo per tutto il corso. Si clona una volta, poi a ogni lezione si aggiorna con git pull.

```
git clone https://github.com/
  mg-i-roc-maz/corsi
cd corsi/microservizi
git pull  # a ogni lezione
```

In ogni cartella di lezione: **slides.pdf**, **start** (si lavora qui), **solution**.

#	Cartella	Stato
1	microservizi/lezione-1	disponibile
2	microservizi/lezione-2	disponibile
3	microservizi/lezione-3	disponibile
4	microservizi/lezione-4	disponibile
5	microservizi/lezione-5	disponibile
6	microservizi/lezione-6	disponibile
7	microservizi/lezione-7	disponibile
8	microservizi/lezione-8	disponibile

Il programma in 8 lezioni

01

Dal monolite ai microservizi

Primo servizio catalog

02

Docker e Compose

Container, PostgreSQL, monolite diviso

03

Comunicazione tra servizi

orders chiama catalog, gateway Nginx

04

Resilienza e configurazione

Timeout, retry, circuit breaker

05

Caching: fondamenti

Redis e cache-aside su catalog

06

Caching nei microservizi

Cache HTTP, invalidazione, k6

07

Architetture event-driven

RabbitMQ, OrderCreated, notifications

08

Pattern avanzati e progetto

DLQ, idempotenza, demo finale

Lezioni 1–2: le fondamenta

LEZIONE 1 · 3 ORE

Dal monolite ai microservizi

TEORIA · 1h15

Monolite vs microservizi, vantaggi e costi, quando non usarli. Bounded context e responsabilità.

LAB · 1h45

Setup dell'ambiente, primo servizio REST catalog, test con REST Client. Un monolite catalog + orders da spezzare.

LEZIONE 2 · 3 ORE

Docker e Docker Compose

TEORIA · 45'

Immagini, container, layer, volumi, reti.

LAB · 2h15

Dockerfile e multi-stage build, Compose con servizio + PostgreSQL, variabili d'ambiente, debug da VS Code. Il monolite diventa due servizi.

Lezioni 3–4: servizi che collaborano

LEZIONE 3 · 3 ORE

Progettare e far comunicare i servizi

TEORIA · 1h15

Database per servizio, API REST e versionamento, API Gateway, comunicazione sincrona.

LAB · 1h45

orders chiama catalog via HTTP, Nginx come gateway, service discovery con il DNS di Compose.

LEZIONE 4 · 3 ORE

Resilienza e configurazione

TEORIA · 1h

Fallimenti parziali, timeout, retry con backoff, circuit breaker, health check, log e correlation ID.

LAB · 2h

Si spegne catalog e si osserva orders. Timeout, retry e circuit breaker con opossum, endpoint /health, log strutturati.

Lezioni 5–6: caching

LEZIONE 5 · 3 ORE

Caching: fondamenti

TEORIA · 1h15

Hit e miss, TTL, cache-aside, write-through, write-behind. Invalidazione, dati stale, cache stampede.

LAB · 1h45

Redis nel Compose, cache-aside su catalog, tempi con e senza cache, esplorazione delle chiavi con redis-cli.

LEZIONE 6 · 3 ORE

Caching nei microservizi

TEORIA · 1h

Cache locale vs distribuita, Cache-Control ed ETag, cache sul gateway, invalidazione tra servizi, hit ratio.

LAB · 2h

Cache HTTP con Nginx, invalidazione quando un prodotto cambia, load test con k6, cache stampede simulato e mitigato.

Lezioni 7–8: event-driven

LEZIONE 7 · 3 ORE

Architetture event-driven

TEORIA · 1h15

Eventi vs comandi, publish/subscribe, RabbitMQ e cenni a Kafka, consistenza eventuale, at-least-once, idempotenza.

LAB · 1h45

RabbitMQ nel Compose, orders pubblica OrderCreated, notifications lo consuma. Code ed exchange nella Management UI.

LEZIONE 8 · 3 ORE

Pattern avanzati e progetto finale

TEORIA · 45'

Saga orchestrata vs coreografata, Outbox, Dead Letter Queue, cenni di observability.

LAB · 1h45 + DEMO · 30'

Retry e DLQ, consumer idempotente, integrazione di gateway, cache ed eventi. Chiusura con le demo dei gruppi.

Prima della lezione 1

1

Installate Docker Desktop e VS Code

Con le estensioni Docker, REST Client e Dev Containers.

2

Clonate github.com/mg-i-roc-maz/corsi

Si lavora in microservizi/lezione-1/start; in solution c'è la soluzione per riallinearsi.

3

Aprite il Dev Container

Ambiente già pronto: la prima ora non va persa in installazioni.

4

Se partite da zero: modulo pre-corso (2h)

HTTP, JSON e riga di comando, facoltativo.

IL METODO

Cosa si rompe se…?

`…catalog si spegne?`

`…Redis perde le chiavi?`

`…un evento arriva due volte?`

Ogni lezione chiude con una domanda così: i trade-off sono la parte che resta.